

SD	Sistemas Distribuidos
26/27	Water Management
	Tutorial: Introducción a Railway en configuración CI/CD

Preámbulo

Objetivo: familiarizarse con Railway antes de usarlo en la práctica Water_Management.

Prerrequisitos

Antes de empezar, el alumno debe conocer y disponer de lo siguiente:

- **Cuenta de GitHub**, activa y con acceso desde el navegador.
- **Cuenta en railway.app**, creada preferiblemente con login de GitHub (facilita la conexión de repos).
- **VS Code instalado**, con el panel de Source Control familiarizado (o Git por línea de comandos, si se prefiere) — se usa para inicializar el repo, hacer commits y publicarlo a GitHub.
- **Python 3 instalado** en el equipo, necesario tanto para el ejemplo del bloque 2 como para el script de prueba de Kafka del bloque 3.
- **Conocimientos básicos de terminal/línea de comandos**: navegar entre carpetas y ejecutar comandos como pip install.
- **Nociones mínimas de Docker**: entender qué es un Dockerfile y para qué sirve (no hace falta tener Docker instalado localmente, ya que Railway construye la imagen en la nube, pero es recomendable).
- **Concepto de variables de entorno** (como PORT), ya que Railway las usa para configurar los servicios en tiempo de ejecución.

1. Qué es Railway

Railway es una plataforma de despliegue en la nube (PaaS) pensada para levantar aplicaciones y servicios sin gestionar servidores manualmente.

Ideas clave:

- **Proyecto** = contenedor lógico donde agrupas varios servicios (por ejemplo: tu app + una base de datos + Kafka). Los servicios de un mismo proyecto se ven entre sí por red privada interna.
- **Servicio** = una pieza desplegada: puede venir de tu propio código (repo de GitHub, con o sin Dockerfile) o de una plantilla ya empaquetada por Railway o la comunidad (bases de datos, Kafka, Redis, etc.).
- **Build automático**: si conectas un repo de GitHub, cada git push reconstruye y redespiega el servicio solo — es el "CD / Continuous Deployment" ya integrado, sin necesidad de configurar nada aparte.
- **Red**: cada servicio tiene una URL/host interno (nombre.railway.internal) para hablar con otros servicios del proyecto sin salir a internet, y opcionalmente un dominio público o TCP Proxy para ser accesible desde fuera (por ejemplo, desde vuestros PCs de laboratorio).

En resumen: Railway sustituye a "tener un servidor propio + Docker + Nginx + CI (Continuous Integration) manual" por una interfaz donde conectas un repo o eliges una plantilla y ya tienes el servicio corriendo y accesible.

2. Desplegar un "Hello World" con CI/CD (VS Code → GitHub → Railway)

2.1 Preparar el proyecto local

Cualquier lenguaje sirve; ejemplo mínimo en Python:

```
# server.py
import os
from http.server import BaseHTTPRequestHandler, HTTPServer

port = int(os.environ.get("PORT", 3000))

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header("Content-Type", "text/plain")
        self.end_headers()
        self.wfile.write(b"Hello World desde Railway!")

print(f"Escuchando en el puerto {port}")
HTTPServer(("0.0.0.0", port), Handler).serve_forever()

# Dockerfile
FROM python:3.12-alpine
WORKDIR /app
COPY . .
EXPOSE 3000
CMD ["python", "server.py"]
```

Importante: En este ejemplo el servidor debe escuchar en el puerto que Railway indique mediante la variable de entorno `PORT`, no en un puerto fijo — Railway lo asigna dinámicamente.

2.2 Subir el código a GitHub desde VS Code

1. Abre la carpeta del proyecto en VS Code.
2. Panel Source Control (icono de rama, Ctrl+Shift+G) → Initialize Repository.
3. Escribe un mensaje de commit (ej. "Hello World inicial") → Commit.
4. Botón Publish Branch → elige Publish to GitHub → repo público o privado, nombre p. ej. railway-hello-world.

VS Code crea el repo en GitHub y sube el código en un solo paso.

2.3 Conectar Railway al repo

1. En railway.app → New Project → Deploy from GitHub repo.
2. Autoriza el acceso a GitHub (solo la primera vez) y selecciona railway-hello-world.
3. Railway detecta el Dockerfile y construye la imagen automáticamente.
4. En Settings → Networking → Generate Domain obtienes una URL pública tipo railway-hello-world-production.up.railway.app.
5. Ábrela en el navegador: debe mostrar "Hello World desde Railway!".

2.4 Probar el ciclo de CI/CD

1. Cambia el texto del mensaje en server.py (por ejemplo, añade tu nombre).
2. En VS Code: Source Control → mensaje de commit → Commit → Sync Changes (esto hace push).
3. En Railway, pestaña Deployments: verás un build nuevo iniciarse solo, sin tocar nada manualmente.
4. Al terminar, recarga la URL pública: debe mostrar el texto actualizado.

Con esto el ciclo VS Code → GitHub → Railway queda demostrado de extremo a extremo.

3. Desplegar Kafka en Railway y probar el acceso remoto

3.1 Desplegar Kafka desde plantilla

1. Dentro del mismo proyecto (o uno nuevo) → + New → Template → busca Kafka.
2. Elige la plantilla oficial apache/kafka (modo KRaft, sin ZooKeeper — más simple de gestionar).
3. Railway despliega el broker y crea automáticamente un volumen persistente para los datos.

3.2 Exponerlo hacia fuera con TCP Proxy

Kafka usa el protocolo TCP nativo, no HTTP, así que no se expone con un dominio público normal:

1. Servicio de Kafka → Settings → Networking → TCP Proxy → + Add.
2. Indica el puerto interno del broker (normalmente 9092).
3. Railway asigna un host y puerto públicos, tipo xxxxx.proxy.rlwy.net:19097 — este es el endpoint que usarán los clientes externos (equipos de laboratorio, vuestro PC, etc.).

3.3 Probar el acceso remoto desde un equipo externo

Desde cualquier PC con Python instalado (no hace falta que sea el mismo donde está Railway):

```
pip install kafka-python

# test_kafka.py
from kafka import KafkaProducer, KafkaConsumer

BOOTSTRAP = "xxxxx.proxy.rlwy.net:19097" # sustituir por el host:puerto real del TCP Proxy

# Productor: enviar un mensaje de prueba
producer = KafkaProducer(bootstrap_servers=BOOTSTRAP)
producer.send("topic-prueba", b"Hola desde un equipo remoto")
producer.flush()
print("Mensaje enviado.")

# Consumidor: leerlo de vuelta
consumer = KafkaConsumer(
    "topic-prueba",
    bootstrap_servers=BOOTSTRAP,
    auto_offset_reset="earliest",
    consumer_timeout_ms=5000
)
for msg in consumer:
    print("Recibido:", msg.value.decode())
```

Si el mensaje enviado se recibe de vuelta, el broker es accesible correctamente desde fuera de Railway — exactamente el mismo mecanismo que se usará después en WM_WS y WM_FO desde los PCs de laboratorio.

3.4 Comprobación alternativa por línea de comandos

Con Kafka instalado localmente (aunque sea solo los binarios, sin arrancar broker propio), también sirve ejecutar los scripts y utilidades que permiten acceder a sus funcionalidades:

```
kafka-topics.sh --bootstrap-server xxxxx.proxy.rlwy.net:19097 --list
```

Debe devolver la lista de topics existentes (vacía si es la primera vez) sin dar error de conexión.